

RELATÓRIO PROJETO FINAL

Por Vicente

Índice:

- Introdução
- Estrutura do programa
 - Janela principal
 - Janela ajuda
 - Janela definições
- Funções
 - Kit
 - Captura
 - Ler
 - Limpeza
 - Abrir
 - Enviar
 - Perfil
 - Desenhara
 - Fechar
- Dificuldades e limitações
 - Dificuldades:
 - Circular Imports
 - Usar ficheiros de texto
 - Limitações:
 - UI Básico
 - Capturas de ecrã
- Arquitetura técnica

Introdução

Este relatório é para documentar o meu projeto final de Python For AI, que é um assistente feito para ser usado por professores e alunos na SharkCoders e outros programadores.

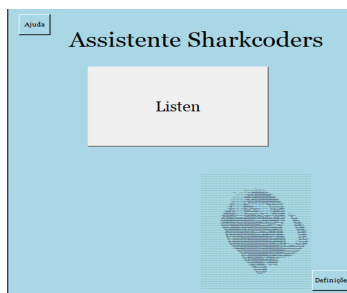
O programa é controlado através de um botão e a voz do utilizador, que diz certas palavras para o microfone, que depois são traduzidas em texto com STT (tecnologia que consegue reconhecer o que uma voz quer dizer e convertê-la em texto em tempo real) e ASR (tecnologia que consegue filtrar a voz e os fonemas de uma língua). O assistente pode reconhecer este texto e mostrá-lo ao utilizador na janela do assistente, e depois usá-lo como indicação para fazer certas ações para ajudar o utilizador.

Estas funções já são pré-definidas e algumas precisam de aceder a recursos externos como o Whatsapp e o Github, através da sua API. As funções que o assistente pode fazer são: abrir um pré-determinado conjunto de aplicações, sendo esse conjunto chamado “Kit de programação” (Kit), tirar uma captura de ecrã do ecrã inteiro e salvá-lo com um nome escolhido pelo utilizador (Captura), ler o texto de uma imagem dada pelo utilizador e mostrá-lo na janela (Leitura), fechar todas as janelas abertas no OS (Limpeza), abrir uma aplicação através da tecla “Windows” (abrir), enviar uma imagem com uma caption a um contacto ou grupo no Whatsapp, tudo escolhido pelo utilizador (Enviar), mostrar dados sobre um perfil Github na janela através do Github API (Perfil) e fechar a si próprio (Fechar).

O assistente também é capaz de outras coisas como responder ao utilizador com a sua voz, perceber se o utilizador pôs informação errada e dar instruções de como usar o assistente em texto.

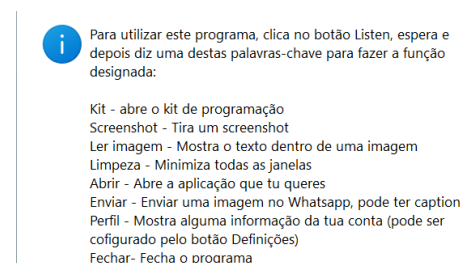
Estrutura do programa (Frontend)

O programa ao abrir começa neste ecrã:

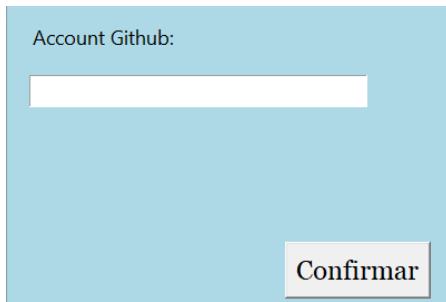


Esta é a janela inicial do assistente. Na janela inicial há dois botões pequenos nos cantos, um botão grande no meio e o senhor tubarão que é a mascote do programa, representado em ASCII.

O botão do canto superior esquerdo, que tem o texto “Ajuda”, mostra uma MessageBox do Tkinter que dá instruções de que funções existem e a palavra-chave necessária para a usar



O botão do canto inferior direito, que tem o texto “Definições”, abre uma segunda janela onde há uma Entry do Tkinter para colocar o Username da conta Github.



O botão de meio branco, que tem o texto “Listen”, é a parte principal do assistente. Ao pressionar o botão, o assistente diz para falar e depois o utilizador pode dizer uma das palavras-chave para ativar uma das funções que são todas detalhadas abaixo.

Para perceber melhor a estrutura, está aqui um diagrama do projeto:

<https://canva.link/rczejj8wr7bl32h>

Funções:

Kit: abre o kit de programação

- Diz que vai abrir o kit
- Consegue o login do utilizador
- Define os caminhos que vai abrir (PyCharm, Github Desktop, Visual Studio Code)
- Verifica se o caminho do Pycharm existe
 - Se sim, abre
 - Se não, o assistente diz que não conseguiu abrir o PyCharm
- Verifica se o caminho do Github Desktop existe
 - Se sim, abre
 - Se não, o assistente diz que não conseguiu abrir o Github Desktop
- Verifica se o caminho do VSCode existe
 - Se sim, abre
 - Se não, o assistente diz que não conseguiu abrir o VSCode

Captura: tira uma captura de ecrã

- Diz que vai tirar o screenshot
- Tira o screenshot
- Pede o nome do screenshot
- O utilizador diz o nome
- O screenshot é gravado com o nome que foi dado pelo utilizador

Ler: extrai o texto de uma imagem

- Pede a imagem
- O utilizador dá a imagem

- Se for válida, continua
- Se não, diz que é invalido e a função acaba
- Define o caminho para o tesseract
- Utiliza o OCR do Tesseract para ler a imagem (OCR é uma tecnologia de visão computacional, um ramo do Machine Learning feito para captar, processar e interpretar imagens e vídeos. Esta tecnologia analisa imagens para separar texto, identificar letras e reconstruir a mensagem em texto)
- Devolve o texto que encontrou

Limpeza: fecha as janelas

- Diz que vai minimizar todas as janelas
- Minimiza todas as janelas

Abrir:

- Pergunta ao utilizador que aplicação quer abrir
- O utilizador diz o que quer abrir
- Pressiona na Windows key através de pyautogui
- Escreve o que o utilizador disse na search bar
- Pressiona na tecla Enter para abrir a aplicação

Enviar:

- Abre uma janela onde o utilizador pode por a imagem e caption que quer enviar e o contacto a que quer enviar
- Recolhe a imagem e o contacto
- Se houver caption:
 - Recolhe a caption
 - Abre o Whatsapp Web
 - Manda a mensagem final
- Se não houver caption:
 - Abre o Whatsapp Web
 - Manda a mensagem final

Perfil: Mostra o perfil Github

- Vai buscar o username Github (pode ser definido por Definições)
- Dá request a informação do perfil através da Github API (API é uma lista de regras que programas usar para comunicar entre si, serve de ponte para esses programas. Pode-se disponibilizar, que é a criação de um sistema de dados para ser acedido e consumo, que é o ato de conectar a esses sistemas de dados)
 - Se acontecer um erro 404, diz que o utilizador não foi encontrado
 - Se acontecer outro erro, para
 - Se não, continua
- Mostra na janela o nome, a bio, o nº de followers, o url para aceder ao perfil e o url para aceder aos repositórios

Desenhar: Converte uma imagem em ASCII

- Pede a imagem
- O utilizador dá a imagem
 - Se for válida, continua
 - Se não, diz que é inválido e a função acaba
- Converte a imagem em ASCII (texto)
- Cria um ficheiro com o texto criado

Dificuldades

Depois de muito trabalho, percebi que algumas coisas eram mais difíceis de fazer e que algumas coisas eram demasiado difíceis de fazer eu sozinho.

Para começar, vou falar sobre as minhas maiores dificuldades, a principal sendo os circular imports. Como o programa usa vários ficheiros, é preciso importar funções para os outros ficheiros como as funções dos botões. Isto não é um problema, mas se uma função for importada, qualquer função que o ficheiro de origem usa também é importada que resulta em mais ficheiros importarem, e se isto correr mal, pode levar a que o programa importe funções num loop, resultando num circular import que causa que o assistente inteiro não sequer começa. Com bom planeamento isto pode ser facilmente prevenido, mas eu não planeei assim tão bem levando a vários casos de circular imports. A única maneira que eu encontrei era simplesmente fazer grandes mudanças na estrutura interna do programa e se isso não funcionasse, punha funções num ficheiro novo isolado, que ou dava muito trabalho ou causava desorganização na estrutura externa. Mas não era a única dificuldade, a outra sendo ficheiros de texto, pois no PyCharm é difícil testar caminhos de ficheiros.

Limitações

Para limitações, há algumas, mas nenhuma muito importante. A primeira é o UI. O UI em si é básico e muito simples, sendo um pouco robótico, pois o UI das janelas, botões, labels, messageboxes e entries são feitas com tkinter, que é básico e não muito bonito. Poderia ter usado IA para fazer a UI, mas isso resultava em ter que arranjar muitos erros que iria demorar muito tempo a fazer. A outra limitação foi a inexistência de screenshots parciais, que estavam a ser desenvolvidos há muito tempo, mas não consegui por causa de circular imports.

Arquitetura técnica

Linguagem: Python 3

Packages: tkinter, speech_recognition, pyttsx3, pyautogui, pytesseract, requests, cv2, pywhatkit,

Estrutura dos ficheiros:

Meu_Projeto/

- |— main.py
- |— gui/
 - |— janela.py
 - |— buttons.py
 - |— entries.py
 - |— labels.py
 - |— messageboxes.py
- |— voz/
 - |— reconhecer_voz.py
 - |— tts.py
- |— automation/
 - |— files_open.py
 - |— os_management.py
 - |— screenshots.py
- |— imagens/
 - |— extracao.py
 - |— procesao.py
 - |— whatsapp.py
- |— api/
 - |— github.py
- |— assets/
 - |— account_github.py
 - |— PyWhatKit.py
 - |— shark.py
- |— utils/
 - |— listen.py
 - |— janela_definicoes.py
 - |— janela_enviar.py

Development time: 3 meses

Comunicações usadas: Github API (TCP Port 443), Whatsapp Web (pywhatkit)

